# Week 1: Introduction to Data Management

## DATA

- Data are Raw Facts
- Data are raw facts and figures that have not yet been processed or analyzed. By themselves, they do not provide much meaning.
- It can be numbers, text, images, sounds, or any other form of information.

## TYPES OF DATA

- **Numerical Data:**
  - Data That Represents Numbers Or Quantities.
  - **Example:** 25, 100, 3.14
- **Textual Data:**
  - Data That Represents Words Or Letters.
  - **Example:** "John", "Apple"
- **Visual Data:**
  - Data That Represents Images Or Graphics.
  - **Example:** Photos, Charts
- **Auditory Data:**
  - Data That Represents Sounds Or Voice.
  - **Example:** Music, Recordings
- **Date/Time Data:**
  - Data That Represents Date And Time.
  - **Example:** 2025-06-14, 10:30 Am

## EXAMPLES OF DATA

- Numbers: 123, 42, 88, 360
- Text: Aa, Anna, Book
- Images: Photo, Diagram, Logo
- Sound: Song, Voice, Computer Alert
- Date/Time: June 14, 2025, 10:30 AM

## CHARACTERISTICS OF DATA

- **Raw**: Data is Unprocess and unorganized
- **Unorganized:** Data by itself has no particular order
- **Facts:** Data represented facts about something
- **Needs Processing:** Data must be processed to become meaningful information.

## IMPORTANCE OF DATA

- Helps in decision-making and planning.
- Supports research and analysis.
- Improves efficiency and performance.
- Serves as the foundation for information and knowledge.

## DATABASE

- A database is a structured way of storing data in tables so that it can be easily accessed, modified, and managed.
- A database is an organized collection of related data that is stored electronically so it can be easily accessed, managed, updated, and retrieved.
- In simple terms, a database is a place where information is stored in an organized way, making it easy to find and use whenever needed.

## KEY FEATURES

- **Organized:** Data is stored in tables with rows and columns.
- **Accessible:** Data can be easily retrieved and searched.
- **Updatable:** Data can be added, modified, or deleted.
- **Secure:** Data is protected from unauthorized access.
- **Shareable:** Multiple users can access the data (with permission).

## HOW A DATABASE WORKS

1. Data Input
2. Stored in Database
3. Process / Query
4. Output / Result

## BENEFITS OF A DATABASE

- **Reduces Data Redundancy:** Data is stored once and used in multiple places.
- **Improves Accuracy:** Reduces errors and ensures consistent data.
- **Saves Time:** Quickly search, retrieve, and manage data.
- **Supports Decision Making:** Provides accurate data for reports and analysis.
- **Ensures Data Integrity:** Maintains correctness and consistency of data.

## WHERE IS IT USED?

- Schools
- Hospitals
- Banks
- Businesses

- Government
- Libraries

## DATABASE MANAGEMENT

- **Database Management is the process of:**
  - creating,
  - organizing,
  - storing,
  - retrieving,
  - updating,
  - securing, and
  - maintaining data in a database using a Database Management System (DBMS).

## CREATE

- Setting up a new database.
- Setting up structures such as tables and fields, where data will be stored, organized, and managed.
- **CREATE DATABASE - SAMPLE EXPLANATION:**
  a. **Type the command**: `CREATE DATABASE SchoolDB;`
  b. Execute the command.
  c. The database is created successfully.

## ORGANIZING

- Organizing a database means arranging and storing information in a clear, logical, and systematic way so that it is easy to find, add, update, delete, and manage whenever needed.
- Instead of keeping all information in one large list, the data is grouped into related tables, with each table containing a specific type of information.
- **NOTES:**
  - Each table has a primary key (PK).
  - Foreign keys (FK) are used to link related tables.
  - This design keeps data organized, reduces duplication, and makes it easier to manage.

## STORING

- Storing data is the process of saving information in a database so it can be accessed, updated, and used whenever needed.
- Instead of keeping information on paper or in separate files, a database stores it electronically in an organized manner.
- Storing data means saving information in a database so it can be used, updated, and retrieved whenever needed.
- **Process:** 1. Enter Data -> Data is saved in the database -> 2. Data Stored.
- **Benefits of Storing Data:**
  - Data is organized and easy to manage.
  - Information can be updated when needed.
  - Data is protected from loss. Multiple users can access the data. Data can be retrieved quickly.

## RETRIEVING

- Retrieving data is the process of finding and displaying information stored in a database.
- Users retrieve data when they need to view specific records or generate reports.
- In simple terms, retrieving data means getting the information you need from the database.
- Retrieving data means finding and displaying information that is already stored in the database.
- **Process:**
  - **User Requests Data:** The user searches for specific information.
  - **Database Processes Request:** The database looks for the matching records. SQL Query: `SELECT * FROM Students WHERE Course = 'BSIT';` The database searches the data and finds the matching records.
  - **Data Retrieved:** The requested data is found and displayed.
  - **Result Shown to User:** The user sees the result of the search.
- **Benefits of Retrieving Data:**
  - Find information quickly.
  - Get accurate data for reports and decisions.
  - Save time and effort.
  - Easy access for authorized users.

## UPDATING

- Updating data is the process of modifying or changing existing information in a database.

- This is done when information changes, such as a person's address, phone number, or course.
- In simple terms, updating data means editing existing records to keep the information accurate and up to date.
- Updating data means making changes to existing information in the database to keep it accurate and up-to-date.
- **Process:**
    - **Select Data:** Choose the record to be updated.
    - **Edit Data:** Make the necessary changes in the fields.
    - **Execute Update:** The database applies the changes.
    - **Data Updated:** The record is updated in the database.
- **Example SQL Command:** `UPDATE Students SET Course = 'BSIT', YearLevel = 3 WHERE StudentID = 1002;`
- **Benefits of Updating Data:**
    - Keeps information accurate and reliable.
    - Reflects the latest changes.
    - Prevents incorrect decisions.
    - Saves time and improves efficiency.
- **Remember:** Always update data carefully to avoid mistakes and maintain data integrity.

## SECURING DATA

- Securing Data Is The Process Of Protecting Information Stored In A Database From Unauthorized Access, Theft, Loss, Or Damage.
- It Ensures That Only Authorized Users Can View, Edit, Or Delete Data.
- In Simple Terms, Securing Data Means Keeping Information Safe And Protecting It From People Who Should Not Have Access To It.
- Securing Data Means Protecting Information In The Database From Unauthorized Access, Loss, Theft, Or Damage.
- **How We Secure Data:**
    - **User Authentication:** Require Username And Password To Ensure Only Authorized Users Can Access The Database.
    - **Access Control:** Give Different Users Different Permissions (View, Insert, Update, Delete).
    - **Data Encryption:** Convert Data Into A Coded Format So It Cannot Be Read By Unauthorized People.
    - **Backup:** Create Regular Backups To Restore Data In Case Of Loss Or Damage.
    - **Audit Trail:** Monitor And Record User Activities To Detect And Prevent Suspicious Actions.
- **Controlled Access:**
    - **Authorized User:** Can Access And Use The Data Based On Permissions.
    - **Unauthorized User:** Access Is Denied.
- **Security:** Prevents Unauthorized Access, Data Breaches, And Data Loss. Data Is Safe, Accurate, And Available Only To Authorized Users.
- **Benefits Of Securing Data:**
    - Protects Data From Unauthorized Access.
    - Maintains Data Accuracy And Integrity.
    - Prevents Data Breaches And Identity Theft.
    - Ensures Data Can Be Recovered If Lost Or Damaged.
    - Builds Trust Among Users And Clients

## MAINTAINING

- Maintaining Data Is The Process Of Keeping The Information In A Database Accurate, Complete, Up To Date, And Working Properly Over Time.
- It Involves Regularly Checking, Updating, Correcting, And Backing Up Data To Ensure The Database Remains Reliable.
- In Simple Terms, Maintaining Data Means Taking Care Of The Database So The Information Stays Correct, Organized, And Useful.
- Maintaining Data Means Keeping The Database Accurate, Consistent, Up-To-Date, And Efficient For Reliable Use.
- **Maintenance Tasks:**

    - **Check Data Accuracy:** Review And Verify Data Regularly To Ensure It Is Correct And Complete.
    - **Remove Duplicates:** Find And Delete Duplicate Records To Keep Data Clean And Consistent.
    - **Update Information:** Update Records When There Are Changes Or New Information.
    - **Backup Data:** Create Regular Backups To Protect Data From Loss Or Damage.
    - **Optimize Performance:** Organize And Optimize The Database For Faster Searching And Processing.
    - **Ensure Data Security:** Monitor Access And Apply Security Measures To Protect Data.
- **The Maintenance Process:**
    - Identify Data Issues (Errors, Duplicates, Missing Data, Outdated Information)
    - Perform Maintenance Tasks
    - Save And Apply Changes
    - Verify And Review The Data
    - Database Remains Accurate And Reliable
- **Well-Maintained Database:** Accurate, Up-To-Date, Consistent, Reliable, Easy To Use.
- **Benefits Of Maintaining Data:**
    - Improves Data Accuracy And Reliability.
    - Ensures Data Is Up-To-Date And Relevant.
    - Enhances Performance And Efficiency.
    - Reduces Errors And Prevents Data Loss.
    - Builds Trust And Supports Better Decisions.
- **Remember:** Regular Maintenance Keeps Your Database Accurate, Secure, And Ready To Support Your Needs.

## Week 2: Database Concepts

### DATA VS. INFORMATION

- **Data:** Raw, Unorganized Facts And Figures Without Context.
- **Processing:** Collecting, Organizing, Analyzing, And Interpreting Data.
- **Information:** Processed, Organized, And Meaningful Data That Provides Context And Answers.

| | Data | Information |
|---|---|---|
| **Nature** | Raw Facts And Figures | Processed And Organized Data |
| **Purpose** | Does Not Provide Context | Provides Context And Meaning |
| **Use** | Difficult To Understand | Easy To Understand And Useful |
| **Role** | Input For Processing | Basis For Decision-Making |
| **Examples** | Numbers, Text, Symbols | Reports, Charts, Summaries, Insights |

### DATABASE

- Central repository of shared data
- Data is managed by a controlling agent
- Stored in a standardized, convenient form
- Requires a Database Management System (DBMS)

### DATA IS MANAGED BY A CONTROLLING AGENT

- In a DBMS, the Database Management System (DBMS) acts as the controlling agent between users/applications and the database.
- **DBMS (Database Management System):**
    - Controls access to data
    - Enforces security and permissions
    - Ensures data integrity
    - Manages concurrent access
    - Handles transactions
    - Provides backup and recovery
- **Why a Controlling Agent is Important:**
    - **Security:** Prevents unauthorized access.
    - **Data Integrity:** Ensures accuracy and consistency of data.
    - **Concurrency Control:** Allows multiple users to access data safely.
    - **Data Independence:** Changes in data structure won't affect applications.
    - **Backup & Recovery:** Protects data from loss or system failures.

### DATABASE SYSTEMS

**Five Major Parts of a Database System**

1. **Hardware:** Physical Devices Used To Run The Database System.
    - *Examples:* Database Server, Computers, Storage Devices (Hdd, Ssd), Network Devices.
2. **Software:** Programs That Manage And Interact With The Database.
    - Operating Systems Software
    - Dbms Software
    - Application Programs And Utility Software
    - ***Examples:*** Database Management System (Dbms) (Mysql, Sql Server, Oracle, Sqlite), Application Programs, Utilities (Backup, Security, Etc.).
3. **People (Users):** People Who Design, Manage, Maintain, And Use The Database System.
    - Systems Administrators
    - **Database Administrators (Dba):** Manages The Database, Security, Backup, And Performance.
    - **Database Designers:** Designs Database Structure And Relationships.
    - **Systems Analysts And Programmers:** Develops Programs That Access And Update The Database.
    - **End Users:** Teachers (Record Grades And Manage Classes), Students (View Grades And Schedules), Registrar (Manages Enrollment And Student Records), Cashier (Handles Payments And Billing).
4. **Procedures:** Rules, Policies, And Step-By-Step Instructions For Operating And Managing The Database System.

    - ***Examples:*** Login Procedure, Data Entry Rules, Backup Schedule, Access Control, Disaster Recovery.
5. **Data:** Raw Facts And Figures Stored In The Database. When Processed, They Become Information.
    - ***Examples:*** Student Records, Grades, Subjects, Attendance, Payments.

**Example Of A Database:** A Database Is An Organized Collection Of Related Data Stored In Tables. Example: School Management System (Schooldb).

- **This Database Contains**: Students, Teachers, Subjects, Enrollments, Grades.
- In This Example, Data Is Organized Into Related Tables. The Relationships Between Tables Help Reduce Redundancy And Make Data Easier To Manage And Retrieve.
- ***Legend:*** `Pk` - Primary Key, `Fk` - Foreign Key.

## FILE SYSTEMS

- File processing system where data are stored for each individual application in an organization.
- Each department or area within an organization has its own set of files, often creating data redundancy and data isolation.
- There is no overall map, plan, or model guided application growth.
- **Problems with File Processing System**
    - **Data Redundancy (Duplicate Data):** Same data is stored in multiple files in different departments. High data redundancy.
    - **Data Isolation:** Each department has its own files. Data cannot be easily shared. Difficult to share data.
    - Data inconsistency
    - Difficult data retrieval
    - Limited data security
    - Difficult to generate reports

- **Solution:** Use a Database Management System (DBMS) to integrate data, reduce redundancy, ensure accuracy, and improve data sharing.

## DISADVANTAGES OF A FILE SYSTEM

File Systems Are Simple To Use But Become Inefficient As Data Grows. These Disadvantages Lead To Data Inconsistency, Errors, And Higher Costs.

**1. Program-Data Dependence:** Data Is Tightly Bound With The Programs That Use It. If The Data Structure Changes, The Program Must Also Be Changed.

- **Example:** Changing The Student Record Format Requires Changes In All Programs That Use It.

**2. Duplication Of Data:** The Same Data Is Stored In Multiple Files In Different Applications, Causing Redundancy And Wasting Storage Space.

- Example: Customer Information Is Stored In Sales, Payroll, And Inventory Files.

**3. Limited Data Sharing:** Data Is Difficult To Share Between Departments. Each Application Has Its Own Files, Making It Hard To Access Data.

- **Example:** Hr Cannot Easily Access Data From The Sales Department.

**4. Lengthy Development Times:** Building New Applications Takes A Long Time Because Files Must Be Created, Designed, And Tested Independently.

- **Example:** Creating A New Report Requires Designing New Files And Writing New Code.

**5. Excessive Program Maintenance:** Many Programs Must Be Updated Whenever There Is A Change In Data, Causing High Maintenance Costs And Errors.

- **Example:** Changing A Field Name Means Updating All Programs That Use It.

## DATABASE MANAGEMENT SYSTEM (DBMS)

- Is a collection of programs that enables users to create and maintain a database; a general purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.
    - Stores data in such a way that it becomes easier to retrieve, manipulate, and produce information.
    - DBMS acts as an intermediary between users/applications and the database.

- It stores data, protects it, and allows users to access and manage it easily and efficiently.

## MAIN FUNCTIONS

- **Defining:** Defines The Structure Of The Database (Tables, Fields, Relationships).
- **Constructing:** Builds And Stores The Data In The Database.
- **Manipulating:** Allows Users To Insert, Update, Delete, And Retrieve Data.
- **Sharing:** Enables Multiple Users And Applications To Access The Data Safely And Efficiently.

## BENEFITS OF USING A DBMS

- **Data Security:** Protects data from unauthorized access.
- **Data Integrity:** Maintains accuracy and consistency.
- **Data Sharing:** Allows multiple users to access data.
- **Reduced Redundancy:** Minimizes duplicate data.
- **Better Performance:** Improves data access and processing.
- **Data Independence:** Changes in data do not affect applications.
- **Faster Data Retrieval**
- **Improved Accuracy**
- **Better Decision Making**

## CHARACTERISTICS OF DBMS

1. **Real-World Entity:**
    - A DBMS Is Based On Real-World Entities.
    - It Stores Information About Real-World Objects (People, Places, Things, Events, Ideas) And Their Relationships. Real-World Entities Are Represented As Tables (Relations) In The Database.
    - These Real-World Entities Are Captured And Represented In The Database As Data So That They Can Be Stored, Managed And Used By Applications.
    - **Key Takeaway:** Dbms Represents Real-World Entities As Tables And Their Data, Making It Easier To Store, Manage, Retrieve, And Use Information In Real-Life Applications.
2. **Relation-Based Tables**:
    - A Dbms Stores Data In Relation (Table) Format. Each Table Has Rows (Records) And Columns (Attributes), And Tables Are Related To Each Other Using Keys.
    - Each Table Represents An Entity And Has A Primary Key (Pk) To Uniquely Identify Each Record. Tables Are Related Using Foreign Keys (Fk).
    - **Key Takeaway**: Relation-Based Tables Are The Foundation Of Dbms. They Organize Data Into Rows And Columns And Link Related Tables Using Keys To Ensure Accuracy, Consistency, And Efficiency.
3. **Isolation Of Data And Application:**
    - In A Dbms, Data Is Stored Independently Of Applications. Changes In The Data Structure Do Not Require Changes In The Applications. This Reduces Maintenance Effort And Improves Flexibility.
    - All Applications Access The Same Central Database. Changes In Data Structure Are Made At The Database Level, So Application Programs Do Not Need To Change.
    - **Benefits:** Separates Data From Applications, Reduces Program Dependency On Data, Easier To Update And Evolve The Database, Improves Data Integrity And Security, Supports Multiple Applications Efficiently.
4. **Less Redundancy:**
    - A DBMS Is Designed To Minimize Duplicate Data. Data Is Stored Only Once In The Database And Shared By Different Applications And Users.
    - Redundancy Wastes Storage Space, Increases The Chance Of Inconsistencies, Makes Data Updates Harder, And Increases Maintenance Cost.
    - **Key Takeaway:** Less Redundancy Means Data Is Stored Only Once, Reducing Duplication, Preventing Inconsistencies, And Making The Database More Efficient, Reliable, And Easier To Maintain.

5. **Consistency:**
    - A Dbms Ensures That The Data In The Database Remains Accurate, Valid, And Consistent Before And After Every Transaction.
    - It Enforces Rules, Constraints, And Relationships. Enforces Data Types (E.G., Age Must Be A Number), Domain Rules (E.G. Age > 0), Entity Integrity (E.G., Primary Key Must Be Unique And Not Null), And Referential Integrity (E.G., Cannot Delete Parent Record If Child Records Exist).
    - **Key Takeaway:** Consistency Ensures That The Database Always Follows The Defined Rules And Constraints. It Keeps The Data Accurate, Valid, And Reliable-Before And After Every Transaction.
6. **Query Language**:
    - A Dbms Provides A Query Language That Allows Users To Easily Retrieve, Insert, Update, And Delete Data From The Database.
    - It Acts As A Communication Bridge Between The User And The Database. A Query Is A Request For Information Or An Instruction To Perform An Action On Data In The Database.
    - **Types:** Ddl (Data Definition Language), Dml (Data Manipulation Language), Dcl (Data Control Language), Tcl (Transaction Control Language).
7. **Acid Properties:** Acid Properties Ensure That Database Transactions Are Processed **Accurately, Reliably, And Consistently.**
    - **A** - Atomicity (All Or Nothing): Either Everything Happens Or Nothing Happens.
    - **C** - Consistency: The Database Must Always Remain Correct And Valid.
    - **I** - Isolation: Transactions Run Independently Without Causing Conflicts.
    - **D** - Durability: Committed Data Is Permanently Saved, Even If A System Failure Occurs.
8. **Multiuser And Concurrent Access:**
    - A Dbms Allows Multiple Users To Access And Work With The Data At The Same Time (Concurrently) Without Interfering With Each Other.
    - The System Ensures Data Integrity, Consistency, And Security.
    - **How Dbms Handles It:** Locking Mechanism, Transaction Management, Scheduling, Isolation.
9. **Multiple Views:**
    - A Dbms Allows Different Users To See The Data In Different Ways (Views) Based On Their Needs And Roles.
    - The Underlying Data Remains The Same, But The Representation Is Different. Each User Sees Only The Data Relevant To Them.
    - Improves Security By Hiding Sensitive Data, Simplifies Data By Showing Only What Is Needed.
10. **Security:**
    - A Dbms Provides Security Mechanisms To Protect Data From Unauthorized Access, Misuse, Modification, Or Loss.
    - It Ensures That Only Authorized Users Can Access The Right Data In The Right Way. Security
    - **Features:** Authentication, Authorization, Access Control, Encryption, Auditing & Logging, Backup & Recovery.

## METADATA

- Descriptions of the properties or characteristics of the data, including data types, field sizes, allowable values, and documentation.

| Field Name | Data Type | Size | Allowable Values | Description |
|---|---|---|---|---|
| StudentID | Integer | 5 | | Unique student number |
| StudentName | Alphanumeric | 30 | | |
| DateAdmitted | Date | | | Start date in school |
| GPA | Decimal | 3 | | Grade Point Average |

## THE DATABASE APPROACH

- An approach where data are logically stored in databases, managed by a database management system.
- A database is designed using data models which define the nature and relationships among data.
- The effectiveness and efficiency of a database is directly associated with the structure of the database.

## ADVANTAGES OF THE DATABASE APPROACH

- Planned data redundancy
- Minimal data duplication
- Improved data consistency
- Program data independence
- Allows data to evolve without changing the application programs
- Reduced program maintenance
- Improved data sharing
- Increased productivity of application development
- Enforcement of standards
- Improved data quality
- Improved data accessibility and responsiveness
- Improved decision support

## Week 3: Database System Concepts and Architecture

### DBMS ARCHITECTURE

- DBMS Architecture is the structure or framework that shows how a Database Management System (DBMS) is organized and how users interact with the database.

### THREE-LEVEL ARCHITECTURE OF DBMS

**1. External Level (View Level)**

- Shows Only The Data Needed By Each User.
- **What Is The External Level?**
  - It Is The View Of The Database Seen By Users.
  - Each User Sees Only The Information They Are Allowed To Access.
  - Helps Protect Sensitive Data And Improves Security.
  - Different Users Have Different Views Of The Same Database.
- **Example:** School Information System (Same Database, Different Views!)
  - **User 1 (Student):**
    - Students See Their Grades.
    - **Can See**: Student Id, Name, Subjects, Grades, Class Schedule.
  - **User 2 (Teacher):**
    - Teachers Manage Their Subjects And Students.
    - **Can See:** Student List, Subjects Taught, Grades, Attendance.
  - **User 3 (Registrar)**:
    - Registrar Handles Enrollment And Records.
    - **Can See:** Student Information, Enrollment Records, Courses, Sections.
  - **User N (Administrator):**
    - Administrator Manages The Entire System.
    - **Can See:** All Data, User Accounts, System Settings, Backups, Reports.
- **Benefits:**
  - Improves Security,
  - Hides Unnecessary Data,
  - Simplifies The System For Users,
  - Provides Customized Views,
  - Supports Privacy And Confidentiality.

**2. Conceptual Level (Logical Level)**

- Defines the overall logical design of the database. Overall logical structure of the entire database.
- **Characteristics:**
  - Defines tables (entities),
  - Defines attributes (columns), D
  - Defines relationships between tables,
  - Defines primary and foreign keys,
  - Defines constraints,
  - Defines business rules,
  - Independent of physical storage,
  - Blueprint of the entire database.
- **Who uses it?**
  - Database Designers,
  - Database Administrators (DBAs),
  - System Analysts,
  - Developers.
- **Example in Real Life:**
  - In a School Information System, the conceptual level defines tables like Students, Courses, Subjects, and Enrollment, including how they are related and the rules that apply to the data.
- **Business Rules (Examples)**:
  - Each student must belong to one course.
  - A course can have many students.
  - A student can enroll in many subjects.
  - A subject can have many students.
- **Constraints (Examples):**
  - StudentID cannot be null,
  - Grade must be between 0 and 100,
  - SubjectCode must be unique.
- **Summary:** The Conceptual Level is the blueprint of the database. It defines WHAT data is stored and HOW the data is related, without describing how the data is physically stored.

**3. Internal level (physical level)**

The internal level describes how data is physically stored and managed in the database. how the data is physically stored on storage devices (files, indexes, storage structures, etc.).

**1. Database files:** Sql server stores the database in physical files.

- **Mdf (main data file):** stores the actual database data. contains: tables, records, indexes.
- **Ldf (log file):** stores every transaction performed in the database. helps recover the database if a problem occurs.

**2. Indexes:** Indexes help sql server find data faster.

- **Without index (slower):** Sql server checks many records one by one. W
- **With index (faster):** Sql server goes directly to the correct record.

**3. Backup:** A backup is a copy of the database. It can be used to restore the database if it is lost or damaged.

- **Example:** if the database is accidentally deleted, it can be restored from the backup.

**4. Encryption:** Encryption protects sensitive data from unauthorized access.

- **Before encryption:** password: admin123.
- **After encryption:** encrypted data: a8#x9p!2lm. Only authorized users with the correct key can read the original data.

5. **Storage summary:** The internal level is responsible for the physical storage of data.

- **Mdf file** (stores actual data),
- **Ldf file** (stores transaction logs for recovery),
- **Indexes (**make searching for data faster),
- **Backup** (creates a copy for recovery),
- **Encryption** (protects data from unauthorized access).
- **In short:** the internal level is the physical storage of the database. It stores data in files, records transactions, uses indexes for speed, creates backups for recovery, and encrypts data for protection.

## SCHEMA

- A schema is the blueprint or structure of a database.
- It defines how the data is organized, including the tables, columns, data types, relationships, and constraints.
- **Schema is of three types:**
    - Physical schema,
    - Logical schema and,
    - View schema.
- **Real-Life Analogy:** SCHEMA (Blueprint) -> DATABASE (Actual House).
- **What the Schema Defines:**
    - **Tables** (The structures that store data),
    - **Columns** (The fields/attributes in each table),
    - **Data Types** (The kind of data such as INT, VARCHAR, DECIMAL, etc.),
    - **Relationships** (How tables are connected using keys),
    - **Constraints** (Rules to ensure data accuracy and integrity).
- **Schema (Structure) vs Database (Actual Data):**
    - **Schema:** The blueprint or design of the database. Defines tables, columns, data types, relationships, and constraints. Created before data is entered.
    - **Database:** The actual collection of stored data. Contains records and information. Filled with data after the schema is created.

## INSTANCES

- Data stored in a database at a particular moment of time
- It contains a snapshot of the database
- A DBMS ensures that its every instance (state) is in a valid state, by diligently following all the validations,constraints, and conditions that the database designers have imposed.
- Each instance must be in a VALID STATE, meaning all validations, constraints, and conditions are satisfied.

## DATABASE INSTANCE

- Database Instance is the actual information stored in the database at a particular point in time.
- **Example:** Class Record Book (Actual Data). This is the current data stored in the Students table for this semester. The data in the table can change over time.
- Instance = Actual Data at a Specific Point in Time.

## DATA MODELS

- **A data model is a plan that shows:**
    - How a database should be designed

    - How data is related
    - How it should be organized
- Example: Student Information System. The data model shows the structure of the database, the relationships between data, and the rules to keep the data accurate and consistent.

## FOUR TYPES OF DATA MODELS

**1. Relational Data Model:**

- Organizes data into tables (relations) consisting of rows and columns. Tables are related to each other using keys.
- **Key Features:**
    - Data is stored in tables (rows and columns).
    - Each table has a Primary Key that uniquely identifies each record.
    - Tables are connected using Foreign Keys.
    - Reduces data redundancy and improves data integrity.
    - Widely used in modern database systems.
- **Key Terms:**
    - **Relation (Table):** A collection of related data organized in rows and columns.
    - **Tuple (Row):** A single record in a table.
    - **Attribute (Column):** A field that represents a property of the data.
    - **Domain:** The set of allowed values for an attribute.
- **Advantages:**
    - Easy to understand and use,
    - Ensures data integrity and consistency,
    - Reduces data redundancy,
    - Supports powerful query language (SQL),
    - Highly flexible and scalable.
- **DBMS That Use Relational Data Model:** MySQL, Microsoft SQL Server, Oracle Database, PostgreSQL, IBM Db2, Microsoft Access, SQLite, MariaDB, SAP HANA, Teradata, Informix, Firebird

**2. Semistructured Data Model**

- The Semistructured Data Model stores data that does not follow a fixed table structure. Each record can have different fields and data types. Commonly used formats like JSON or XML.
- **Key Features:**
    - Data does not follow a fixed schema.
    - Each record can have different fields.
    - Supports nested and hierarchical data.
    - Commonly uses JSON, XML, YAML, or BSON.
    - Flexible and easy to evolve as data changes.
- **How it Works:**
    - Data is stored without a fixed table structure.
    - Tags, keys, or labels are used to identify and organize data.
    - Applications can easily read and process the data.
- **Advantages:**
    - Flexible schema,
    - Handles complex and nested data,
    - Easier and faster to develop applications,
    - Scalable for large volumes of data,
    - Cost-effective for modern applications,
    - Easy to store and scale, Great for evolving data.
- **Common Use Cases (Real-World Examples):** Web content management, Mobile applications, Product catalogs, User profiles, IoT and sensor data, Social media data, Log Files, Email Data.
- **Common Technologies (DBMIS):** MongoDB, CouchDB, Firebase Firestore, Apache Cassandra, Couchbase, Realm Database, Amazon DynamoDB, Google Cloud Firestore, OrientDB, RavenDB, MarkLogic, ArangoDB

## SIDE BY SIDE COMPARISION

| Feature | Semi-Structured Data | Structured Data (Relational) |
|---|---|---|
| Structure | No fixed structure (uses keys/tags) | Fixed structure (rows & columns) |
| Flexibility | Highly flexible | Less flexible |
| Data Consistency | May vary between records | Consistent |

| | | |
|---|---|---|
| Querying | More complex (NoSQL queries) | Easy with SQL |
| Best Use | Evolving, varied, unstructured data | Transactional, reporting, business data |
| Examples | JSON, XML, Logs, IoT, Social Media | MySQL, PostgreSQL, SQL Server, Oracle |

**3. Entity-Relationship (ER) Data Model**

- The Entity-Relationship Data Model is a conceptual data model used to design database structures. It represents data as entities, their attributes, and the relationships between entities using an ER Diagram. The ER Model is a high-level conceptual model used to design databases before implementing them in any DBMS.
- **Key Features:**
  - Represents real-world objects as entities.
  - Defines relationships between entities.
  - Describes properties of entities as attributes.
  - Uses keys to uniquely identify entities.
  - Helps in database design before implementation.
- **ER Diagram Components:**
  - **Entity (Rectangle):** Represents an entity (a real-world object).
  - **Relationship (Diamond):** Represents a relationship between entities.
  - **Attribute (Oval):** Represents a property or characteristic.
  - **Key Attribute (Underlined Attribute):** Uniquely identifies each entity.
  - **Line:** Connects entities to relationships and attributes.
- **Uses / Benefits:**
  - Provides a clear visual design of database structure.
  - Helps identify entities, relationships, and constraints.
  - Improves communication between users and developers.
  - Reduces errors in database design.
- **Commonly Used In:**
  - Database design and planning,
  - Requirements analysis,
  - System documentation,
  - Communication with stakeholders.
- **Used By Tools:** MySQL Workbench, Microsoft Visio, Draw.io / diagrams.net, ER/Studio, Lucidchart.
- **Popular DBMS That Use ER Model (For Design Purposes):** MySQL, PostgreSQL, Microsoft SQL Server, Oracle, SQLite, MariaDB, IBM Db2, Microsoft Access, SAP HANA, Teradata, Informix, Firebird.

**4. Object-Based Data Model**

- The Object-Based Data Model represents data as objects, which are similar to real-world objects. An object has both data (attributes) and behavior (methods).
- **Explanation:**
- Extends the object-oriented programming concept to databases.
- Data is represented as objects.
- **Each object has:**
  - Attributes (data),
  - Methods (operations or functions).
- Supports complex data types, inheritance, encapsulation, and reusability.
- Commonly used in Object-Oriented DBMS.
- **Example (Real-World):** Consider an EMPLOYEE object in a company. Each employee has data (attributes) and can perform actions (methods). A class is a blueprint. Objects are instances created from the class, each having its own data values and behavior.
- **Key Point:**
  - Object-Based Data Model is powerful for handling complex data, real-world relationships, and reusable code (methods).
  - It is the foundation of modern Object-Oriented Databases.
- **DBMS That Use Object-Based Data Model:**
- ObjectDB (Pure object-oriented database), db4o (Object database for .NET, Java), V
- **VERSANT** (High-performance object database),
- **GemStone/S** (Scalable object database). Other
- **Examples:** Objectivity/DB, O2 (Object-Oriented DBMS), PostgreSQL

(supports object types & methods), Oracle (Object-Relational features).

## DATA INDEPENDENCE

- The capacity to change the schema at one level of a database system without having changed the schema at the next higher level.
- **In Simple Terms:** You can change the structure (schema) at a lower level of the database system without affecting the structure (schema) at the next higher level.

## LOGICAL DATA INDEPENDENCE

- The ability to change the conceptual schema (logical structure of the database) without changing the external schemas (user views) or application programs.
- **Key Point:**
  - Changes occur in the CONCEPTUAL LEVEL only.
  - EXTERNAL LEVEL remains the same.
- **Example Scenario:** A school database stores student information. The database designer decides to add a new attribute (Email) to the STUDENT table in the conceptual schema. The student portal (users' view) and the application programs do not need to change.
- **Benefits:**
  - Easier to modify database structure,
  - No changes needed in user views,
  - Application programs remain same,
  - Improves flexibility and maintainability.
- **Summary:** The logical schema was changed by adding a new attribute (Email), but the user view (external schema) and application programs continue to work without any modification. This is **Logical Data Independence.**

## PHYSICAL DATA INDEPENDENCE

- The ability to change the internal (physical) storage structure of the database without changing the conceptual schema (logical structure) or external schemas (user views).
- **Key Point:**
  - Changes occur in the INTERNAL LEVEL only.
  - CONCEPTUAL LEVEL and EXTERNAL LEVEL remain the same.
- **Example Scenario:** A school database is running on a traditional hard drive (HDD). The database administrator decides to improve performance by: Adding indexes, Compressing data, Moving the database to an SSD, Enabling encryption. Despite these physical changes, the conceptual schema and user views remain exactly the same. Users and applications continue to work without any modification.
- **Benefits:**
  - Improves performance and efficiency,
  - Enhances security and reliability,
  - No impact on user views,
  - No changes needed in applications,
  - Easier system maintenance.
- **Summary:** The physical storage was improved (HDD to SSD, indexing, compression, encryption), but the conceptual schema and user views remain the same. Users and applications continue to work without any changes. This is Physical Data Independence.

## DBMS LANGUAGES

- DBMS Languages are a set of commands used to communicate with a Database Management System (DBMS).
- They allow users and database administrators to create databases, store data, retrieve information, modify records, control user access, and manage transactions.
- A DBMS has appropriate languages and interfaces to express database queries and updates. Database languages can be used to read, store and update the data in the database.

| Language | Purpose | Example Commands | Operation |
|---|---|---|---|
| **DDL** (Data Definition Language) | Define and modify database structure (schema) | CREATE, ALTER, DROP, TRUNCATE, RENAME | Creates, alters, or deletes objects |
| **DML** (Data Manipulation Language) | Read, store, update, and delete data | SELECT, INSERT, UPDATE, DELETE | Retrieves and modifies data |

| | | | |
|---|---|---|---|
| **DQL** (Data Query Language) | Used to retrieve or view data from the database. | SELECT | Query |
| **DCL** (Data Control Language) | Control access and permissions | GRANT, REVOKE | Manages user rights and access |
| **TCL** (Transaction Control Language) | Manage transactions and ensure integrity | COMMIT, ROLLBACK, SAVEPOINT | Ensures data is correct and consistent |

- **IN SHORT:** DDL defines the structure, DML modifies the data, DQL queries the data, DCL controls access, and TCL manages transactions.

**1. Data Definition Language (Ddl)**

- DDL statements are used to define, modify, and manage the structure of database objects. It does not affect the data stored in the database.
- DDL commands affect the structure (schema) of the database, not the data inside it.
- **Examples of objects managed by DDL:**
  - TABLES, VIEWS, DATABASES, INDEXES.
- **`CREATE:`**
  - It is used to create objects in the database.
  - CREATE builds a new object in the database.
- **`ALTER:`**
  - It is used to alter or modify the structure of the database.
  - ALTER changes the structure of an existing object.
- **`DROP:`**
  - It is used to delete objects from the database.
  - DROP removes the object permanently.
- **`TRUNCATE:`**
  - It is used to remove all records from a table.
  - Removes all records from a table while keeping the table structure.
  - **Tip:** `TRUNCATE` is faster than `DELETE` for removing all rows because it does not log individual row deletions.
- **`RENAME:`**
  - It is used to rename an object.
  - Changes the name of a database object without affecting its data.

**2. Data Manipulation Language (DML)**

- DML statements are used to add, modify, and delete data within database tables.
- **`INSERT:`**
  - It is used to add new records into a database table.
  - INSERT adds news records to a table.
- **`UPDATE`**:
  - It is used to modify existing records in a database table.
  - UPDATE modifies existing records in a table
- **`DELETE:`**
  - It is used to remove one or more records from a table.
  - DELETE removes one or more records from a table.
- **Note:**
  - DML commands manipulate the data stored in tables.
  - They do not change the database structure.

**3. Data Query Language (DQL)**

- DQL statements are used to retrieve data from database tables.
- It is used to retrieve data from one or more database tables.
- SELECT retrieves data based on the specified columns and conditions.
- DQL is read-only.
- IT does not add, modify, or delete data.

**4. Data Control Language (DCL)**

- DCL statements are used to control access and permissions to database objects.
- DCL helps maintain database security and control.
- **`GRANT:`**
  - It is used to give user access privileges to a database.
- **`REVOKE:`**

    - It is used to take back permissions from the user.
- **Common Privileges:**
    - **`SELECT`** (View data),
    - **`UPDATE`** (Modify data),
    - **`DELETE`** (Remove data),
    - **`INSERT`** (Add data),
    - **`ALL PRIVILEGES`** (All of the above).

**5. Transaction Control Language (TCL)**

- TCL is used to manage database transactions. It allows you to save or undo or create points to return to if needed.
- **`COMMIT:`**
    - Permanently saves all changes made during the current transaction.
    - **Simple Meaning:** "Save the changes."
    - **Analogy:** You click Save, your work is permanently saved.

- **`ROLLBACK:`**
    - Cancels all changes made during the current transaction and restores the database to its previous state.
    - **Simple Meaning:** "Undo the changes."
    - **Analogy:** You accidentally delete an important paragraph. You press Undo (Ctrl+Z).
- **`SAVEPOINT:`**
    - Creates a checkpoint within a transaction so you can roll back only to that point instead of canceling the entire transaction.
    - **Simple Meaning:** "Create a checkpoint that you can return to later."
- **Note for SQL Server:**
    - SQL Server uses SAVE TRANSACTION instead of the keyword SAVEPOINT, but both serve the same purpose.
- **KEY TAKEAWAY (Languages):**
    - DBMS languages provide a complete set of commands to define the structure of the database, manipulate the data, control access, and manage transactions.
    - These operations allow us to read, store, and update the data efficiently and securely.

## DBMS CARDINALITIES (RELATIONSHIP CARDINALITY)

Cardinality describes how many instances (records) of one entity can be associated with instances of another entity in a database.

1. **One-To-One (1:1):** One record in Table A is related to only one record in Table B, and vice versa. Example: Person - Passport.
2. **One-To-Many (1:M):** One record in Table A can be related to many records in Table B, but each record in Table B belongs to only one record in Table A. Example: Department - Employees. Real-world: One department has many employees, One teacher teaches many students, One customer places many orders.
3. **Many-To-One (M:1):** Many records in Table A belong to only one record in Table B. (Reverse view of One-to-Many). Real-world: Many employees belong to one department, Many products belong to one category, Many orders are placed by one customer.
4. **Many-To-Many (M:N):** Many records in Table A can be related to many records in Table B. This relationship requires a junction (bridge) table. Example: Students - Courses. Real-world: Students - Courses, Doctors - Patients, Products - Orders.

***NOTE:*** Cardinalities are used when designing Entity-Relationship Diagrams (ERDs) to define how entities are related to each other in a database.

## Week 4: Erd Entity Relationship (Er) Model

### ENTITY RELATIONSHIP (ER) MODEL

- **Definition:**
  - An Entity Relationship (ER) Model is a high-level conceptual data model used to analyze and design a database. It identifies entities, attributes, and relationships before creating the actual database.
- **Purpose/Benefits:**
  - An Entity Relationship (ER) Model is a high-level conceptual data model used to analyze and design a database.
  - It identifies entities, attributes, and relationships before creating the actual database.
- **Real-World Example: School Information System**
  - **STUDENTS:** Student details are stored in the system.
  - **REGISTRAR:** Manages student records and enrollments.
  - **ENROLLS:** Students enroll in various courses.
  - **COURSES:** Course information is stored in the system.
  - **DATABASE**: All data is stored and related in one place.
  - **School Information System:** Students enroll in courses, creating relationships that are modeled using an ER Diagram
- **ER Model Legend**
  - **Entity:** A person, place, object, or concept about which data is stored.
  - **Relationship:** Association between two or more entities.
  - **Attribute:** A property or characteristic of an entity.
  - **1 : M Cardinality:** Defines the relationship between entities.
  - **KEY TAKEAWAY:** An ER Model provides a visual representation of data requirements and relationships, making database design easier, organized, and more accurate.

### COMPONENTS OF ER DIAGRAM

- An ER Diagram is made up of t**hree (3) main components:** ENTITIES, ATTRIBUTES, and RELATIONSHIPS.

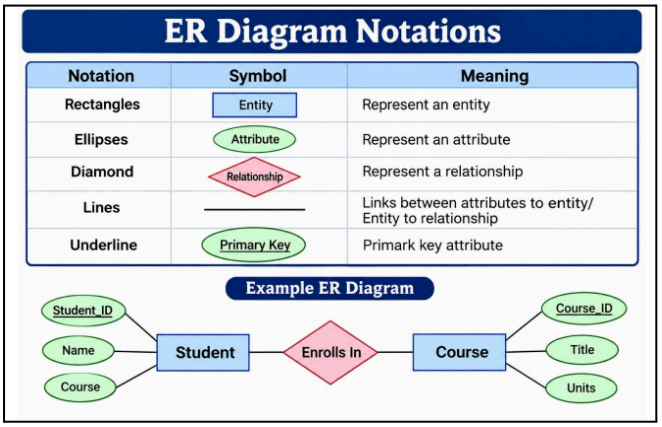
  **1. Entities**
  - Entities are objects or things in the real world that are important to the organization.
  - **Represented by:** RECTANGLE
  - **Example:** STUDENT

  **2. Attributes**
  - Attributes are properties or characteristics that describe an entity.
  - **Represented by:** OVAL
  - **Example**: Student_ID, Name, Email

  3. **Relationships**
  - Relationships define how entities are associated with one another.
  - **Represented by:** DIAMOND
  - **Example:** ENROLLS

- **Sample Er Diagram & Key Points**
  - **Sample:** STUDENT (Student_ID, Name, Email, Date_of_Birth, Phone, Address) - M ENROLLS N - COURSE (Course_ID, Course_Name, Units, Description). A student can enroll in many courses. A course can have many students. (M : N Relationship).
  - Entities are the main objects.
  - Attributes describe the entities.
  - Relationships show how entities are connected.
  - Cardinality defines the nature of the relationship
- **Types of Relationships**
  - **1 : 1** -> One to One
  - **1 : N** -> One to Many
  - **M : N** -> Many to Many
- **Why It Matters:**
  - Understanding these components helps in designing a database that accurately represents real-world data and business rules.

- **Legend:**
  - **Entity:** A person, place, object, or concept about which data is stored.
  - **Attribute:** A property or characteristic of an entity.
  - **Relationship:** Association between two or more entities.
  - **M : N Cardinality:** Defines the number of instances of one entity that can be associated with another.

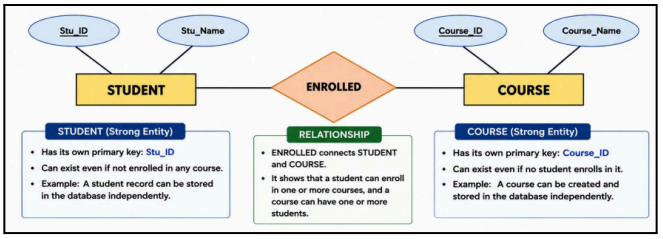



## ENTITIES

- An Entity is a person, place, thing, event, or concept that has independent existence and can be uniquely identified.
- **Examples Of Entities**
  - **Person:**
    - A human being considered as an entity.
    - **Examples:** Student, Teacher, Customer, Employee.
  - **Place:**
    - A location or area considered as an entity.
    - **Examples:** School, Department, Store, City.
  - **Thing:**
    - A physical object considered as an entity.
    - Examples: Computer, Book, Product, Vehicle.
  - **Event:**
    - An activity or incident considered as an entity.
    - **EXAMPLES:** Enrollment, Payment, **Order,** Graduation.
  - **Concept:**
    - An idea or notion considered as an entity.
    - **Examples:** Course, Subject, Membership, Category.
- **Example in ER Diagram:** STUDENT. "STUDENT" is an entity because it refers to a person (a student) with independent existence.
- **Key Takeaway:** Entities are the main data objects in a system. They become the building blocks of an ER Diagram.

## STRONG ENTITY

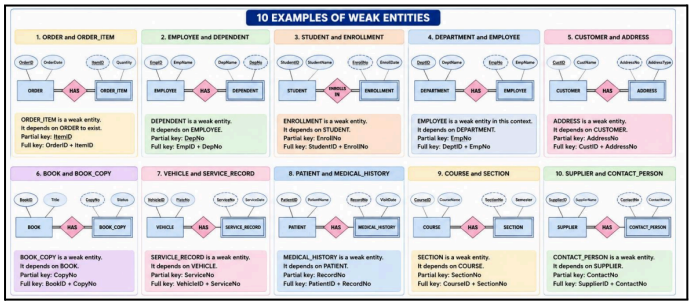
- A STRONG ENTITY is an entity that has its own primary key and can exist independently of other entities.
- **Examples Of Strong Entities**
  - **STUDENT:** Student_ID (Primary Key).
    - A student exists independently in the system.
  - **EMPLOYEE:**
    - Employee_ID (Primary Key).
    - An employee exists independently in the organization.
  - **COURSE:**
    - Course_ID (Primary Key). A
    - course exists independently in the system.
  - **PRODUCT:**
    - Product_ID (Primary Key).
    - A product exists independently in the inventory.




- **KEY TAKEAWAY:** Strong entities have their own primary key and can exist without depending on other entities. They are represented by rectangles in an ER Diagram. Strong entities form the core of the database and are the building blocks of relationships

## WEAK ENTITY

- A weak entity is an entity that cannot be uniquely identified by its own attributes.
- It depends on another entity (strong entity) for its existence and identification
- A weak entity is an entity that does not have a primary key of its own.
- It cannot be uniquely identified by its own attributes.
- It depends on a strong (owner) entity for its existence and identification through an identifying relationship.
- **Example: Employee And Dependent**
    - **EMPLOYEE** (Strong Entity - Has Its Own Primary Key: Empid). Attributes: Empid, Empname.
    - **HAS** (Identifying Relationship).
    - **DEPENDENT** (Weak Entity - Depends On Strong Entity). Attributes: Depname, Depno (Partial Key).
- **When To Use A Weak Entity?**
    - When An Entity Does Not Have Enough Attributes To Form A Primary Key.
    - When The Entity's Existence Depends On Another Entity.
- **Key Takeaway:** A weak entity (double rectangle) is always connected to a strong (owner) entity through an identifying relationship (double border for weak entity + dashed line).
- **Key Rules:**
    - A weak entity has no primary key of its own.
    - It has a partial key (discriminator) shown with a dashed underline.
    - It has total participation in its identifying relationship (double line).
    - It cannot exist without the owner entity. It is identified by: (Owner Key + Partial Key)




- **Key Takeaway:** A weak entity is always connected to a strong (owner) entity through an identifying relationship. It is represented by a double rectangle and uses a partial key (dashed underline).

## ATTRIBUTES

- An attribute is a property or characteristic that describes an entity or a relationship.
- In ER diagrams, attributes are represented by ovals (ellipses) connected to entities or relationships.
- **Types Of Attributes**
    - Simple Attribute: Cannot be divided (e.g., Gender)
    - **Composite Attribute:** Can be divided into smaller parts (e.g., Name)
    - **Single-valued Attribute:** Has only one value (e.g., DateOfBirth)
    - **Multi-valued Attribute**: Has multiple values (e.g., PhoneNo)
- **Examples Of Attribute Types**
    - **Simple Attribute:** STUDENT with attributes StudentID, Age, GPA, Gender. Age, Gender, and GPA cannot be divided further. They are simple attributes.
    - **Composite Attribute:** STUDENT with attribute Name. Name can be divided into FirstName, MiddleName, and LastName.
    - **Single-Valued Attribute**: STUDENT with attributes StudentID, Email, DateOfBirth, Gender. Email, DateOfBirth, and Gender have only one value.
    - **Multi-Valued Attribute:** STUDENT with attribute PhoneNo. PhoneNo can have multiple values (e.g., 09171234567, 09981234567).

## OTHER COMMON ATTRIBUTE EXAMPLES

1. **DERIVED ATTRIBUTE:** STUDENT with attributes DateOfBirth and Age. Age is derived from DateOfBirth. It can be computed.
2. **KEY ATTRIBUTE:** STUDENT with attribute StudentID. StudentID uniquely identifies each student. It is the primary key.

3. **OPTIONAL/NULLABLE ATTRIBUTE:** STUDENT with attributes Email and NickName. NickName is optional and may have NULL (no value).
4. **COMPLEX ATTRIBUTE (Composite + Multi-valued):** STUDENT with attribute Address. Address is multi-valued (a student may have multiple addresses) and composite (made up of HouseNo, Street, City, ZipCode).
5. **RELATIONSHIP ATTRIBUTE:** STUDENT ENROLLS IN COURSE. EnrollDate describes the relationship between STUDENT and COURSE.
6. **ATTRIBUTE WITH DOMAIN:** STUDENT with attribute YearLevel. YearLevel has a domain of allowed values: 1, 2, 3, or 4.

**KEY TAKEAWAYS:**

- Attributes describe entities or relationships.
- They can be simple, composite, single-valued, multi-valued, derived, etc.
- Key attributes uniquely identify each entity.

## RELATIONSHIP

- A relationship is an association or connection between two or more entities that describes how they interact with each other in a database.
- **KEY POINTS**
  - Represented by a diamond in an ER Diagram.
  - Connects two or more entities.
  - Describes how entities interact or are associated.
  - Can have different cardinalities: 1:1 (One-to-One) | 1:M (One-to-Many) | M:N (Many-to-Many)

## SAMPLES OF RELATIONSHIPS

1. **STUDENT ENROLLS IN COURSE**: M : N. A student can enroll in many courses. A course can have many students. (Many-to-Many).
2. CUSTOMER PLACES ORDER: 1 : M. A customer can place many orders. An order is placed by one customer. (One-to-Many).
3. **EMPLOYEE WORKS IN DEPARTMENT:** M : 1. An employee works in one department. A department can have many employees. (Many-to-One).
4. **DOCTOR TREATS PATIENT:** 1 : M. A doctor can treat many patients. A patient is treated by one doctor. (One-to-Many).
5. **AUTHOR WRITES BOOK:** 1 : M. An author can write many books. A book is written by one author. (One-to-Many).
6. **SUPPLIER SUPPLIES PRODUCT:** 1 : M. A supplier can supply many products. A product is supplied by one supplier. (One-to-Many).
7. **TEACHER TEACHES CLASS:** 1 : M. A teacher can teach many classes. A class is taught by one teacher. (One-to-Many).
8. **PASSENGER BOOKS FLIGHT:** M : N. A passenger can book many flights. A flight can be booked by many passengers. (Many-to-Many).
9. **USER OWNS ACCOUNT:** 1 : 1. A user owns one account. An account is owned by one user. (One-to-One).
10. **LIBRARY MEMBER BORROWS BOOK:** M : N. A library member can borrow many books. A book can be borrowed by many members. (Many-to-Many).

**REMEMBER:** Relationships help us understand how data in different entities are connected in the real world.

## DIFFERENCE BETWEEN "M" AND "N"

- In ER diagrams, relationships show how entities are connected. The letters "M" and "N" are used to express many.
- **VISUAL EXPLANATION**
  - **1:M (One-to-Many)**: One entity is related to many of another entity.
  - **M:N (Many-to-Many):** Many entities are related to many of another entity.

**M vs N Comparison**

| | M (Many) | N (Many) |
|---|---|---|
| Meaning | Represents many occurrences of one entity. | Represents many occurrences of another entity. |
| Use | Used when one entity can be associated with many of another entity. | Used when an entity can be associated with many of the other entity. |
| Cardinality Type | Used in 1:M (One-to-Many) relationships. | Used in M:N (Many-to-Many) relationships. |

| | M (Many) | N (Many) |
|---|---|---|
| Example | One Department has many Employees. | Many Students enroll in many Courses. |
| How it works | "M" is on one side, "M" on the many side. | "M" is on one side and "N" on the other side. |

**M vs N Examples**

- **1 : M (DEPARTMENT HAS EMPLOYEE):**
  - **Example:** One department has many employees. A department can have many employees. Each employee belongs to one department only.
  - **Real-life Example**: HR Department has many employees.
- **M : N (STUDENT ENROLLS IN COURSE):**
  - **Example:** Many students enroll in many courses. A student can enroll in many courses. A course can have many students.
  - **Real-life Example:** Students enroll in multiple courses, and each course has many students.

## SUMMARY TABLE

| Relationship Type | Notation | Read As | M and N Position |
|---|---|---|---|
| **1:M (One-to-Many)** | 1 -- M | One to Many | 1 on one side, M on the many side. |
| **M:N (Many-to-Many)** | M -- N | Many to Many | M on one side, N on the other side. |

## MORE EXAMPLES

1. **1:M (TEACHER TEACHES CLASS):** One teacher teaches many classes. Each class is taught by one teacher. Real-life: A teacher handles many classes.
2. **M:N (AUTHOR WRITES BOOK):** Many authors can write many books. Many books can be written by many authors. Real-life: Authors write multiple books, books have multiple authors.
3. **1:M (CUSTOMER PLACES ORDER):** One customer places many orders. Each order is placed by one customer. Real-life: A customer can place multiple orders.
4. **M:N (EMPLOYEE SKILL PROJECT):** Employees have many skills. Projects require many skills. Real-life: Employees use many skills in many projects.
5. **1:M (SUPPLIER SUPPLIES PRODUCT):** One supplier supplies many products. Each product is supplied by one supplier. Real-life: A supplier provides many products.

**KEY TAKEAWAY:**

- M means "many" on one side of the relationship.
- N means "many" on the other side.
- **1:M** = One-to-Many (one side has one, other side has many).
- M:N = Many-to-Many (both sides have many).
- **QUICK MEMORY TIP:** Think of M as "Many here!" and N as "Many there!".

## TYPES OF RELATIONSHIP (In ER Diagrams)

A relationship in an ER Diagram shows how two entities are associated with each other. It defines the cardinality or number of instances of one entity that can be related to instances of another entity.

**1. ONE-TO-ONE (1:1)**

- One instance of Entity A is associated with one and only one instance of Entity B, and vice versa.
  - **EXAMPLES**: Person <-> Passport, Employee <-> ID Card, Car <-> Vehicle Registration.
- **EXAMPLE 1:** Person and Passport (One-to-One): 1 PERSON HAS 1 PASSPORT. A person has one passport. A passport belongs to one person.
- **EXAMPLE 2:** Employee and ID Card (One-to-One): 1 EMPLOYEE ISSUED TO 1 ID CARD. An employee has one ID card. An ID card is issued to one employee.
- **EXAMPLE 3:** Car and Vehicle Registration (One-to-One): 1 CAR REGISTERED AS 1 REGISTRATION. A car has one registration. A registration record belongs to one car.
- **KEY TAKEAWAY:** In a One-to-One (1:1) relationship, each record in one entity is linked to exactly one record in another entity, and each record in the other entity is linked back to exactly one record.

**2. ONE-TO-MANY (1:N)**

- One instance of Entity A can be associated with one or many instances of Entity B, but each instance of Entity B can be associated with only one instance of Entity A.

- **EXAMPLE 1:** Department and Employee: 1 DEPARTMENT HAS N EMPLOYEE. One department can have many employees. Each employee belongs to one department.
- **EXAMPLE 2:** Teacher and Student: 1 TEACHER TEACHES N STUDENT. One teacher can teach many students. Each student is taught by one teacher.
- **EXAMPLE 3:** Customer and Order: 1 CUSTOMER PLACES N ORDER. One customer can place many orders. Each order is placed by one customer.

**3. MANY-TO-ONE (N:1)**

Many instances of Entity A can be associated with one instance of Entity B, but each instance of Entity B can be associated with many instances of Entity A.

- **EXAMPLE 1:** Student and Class: N STUDENT ENROLLED IN 1 CLASS. Many students can be enrolled in one class. Each class can have many students.
- **EXAMPLE 2:** Order and Customer: N ORDER PLACED BY 1 CUSTOMER. Many orders can be placed by one customer. Each customer can place many orders.
- **EXAMPLE 3:** Book and Publisher: N BOOK PUBLISHED BY 1 PUBLISHER. Many books can be published by one publisher. Each publisher can publish many books.

**4. MANY-TO-MANY (M:N)**

Many instances of Entity A can be associated with many instances of Entity B, and vice versa. EXAMPLES: Student <-> Course, Doctor <-> Patient, Product <-> Order.

- **EXAMPLE 1:** Student and Course: M STUDENT ENROLLS IN N COURSE. A student can enroll in many courses. A course can have many students.
- **EXAMPLE 2**: Doctor and Patient: M DOCTOR TREATS N PATIENT. A doctor can treat many patients. A patient can be treated by many doctors.
- **EXAMPLE 3:** Product and Order: M PRODUCT INCLUDED IN N ORDER. An order can contain many products. A product can be included in many orders.
- **KEY TAKEAWAY:** A Many-to-Many (M:N) relationship means that both entities can have multiple related records. In relational databases, this relationship is usually implemented using a junction (bridge) table.

## STEPS TO CREATE ER DIAGRAM

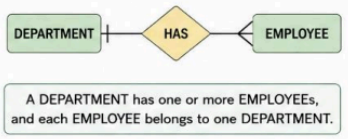
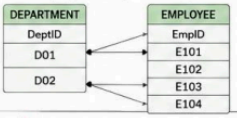
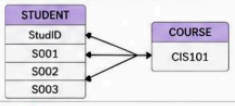
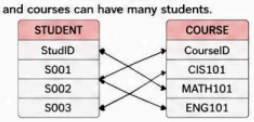
1. **Entity Identification:** Identify the entities involved.
2. **Relationship Identification:** Determine how entities interact.
3. **Cardinality Identification:** Define the numeric relationships.
4. **Identify Attributes:** Detail the properties of entities.
5. **Create ERD:** Draw the diagram.

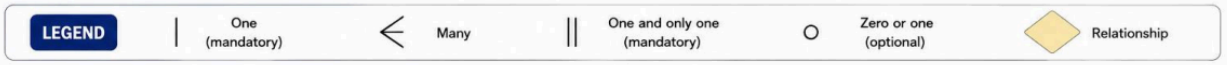
## LET'S GIVE IT A TRY!

**Scenario**: In a university, a Student enrolls in Courses. A student must be assigned to at least one or more Courses. Each course is taught by a single Professor. To maintain instruction quality, a Professor can deliver only one course

- **STEP 1:** ENTITY IDENTIFICATION
- **STEP 2**: RELATIONSHIP IDENTIFICATION
- **STEP 3:** CARDINALITY IDENTIFICATION
- **STEP 4:** IDENTIFY ATTRIBUTES
- **STEP 5:** CREATE THE ERD

# Type of Relationship Notations (Crow's Foot Notation)

| TYPE | NOTATION | MEANING | SAMPLE DIAGRAM | SAMPLE EXAMPLE |
|---|---|---|---|---|
| **1. One to one (1 : 1)** |  | • One instance on A is related to one and only one instance on B.<br>• One instance on B is related to one and only one instance on A. |  <br>Each PERSON has exactly one PASSPORT, and each PASSPORT belongs to one PERSON. | **Example:**<br>A person has one passport, and a passport is issued to one person.<br>  |
| **2. One to many (1 : N)** |  | • One instance on A is related to one or more instances on B.<br>• Each instance on B is related to one and only one instance on A. |  <br>A DEPARTMENT has one or more EMPLOYEEs, and each EMPLOYEE belongs to one DEPARTMENT. | **Example:**<br>A department has many employees, but each employee belongs to one department.<br>  |
| **3. Many to one (N : 1)** |  | • Many instances on A are related to one instance on B.<br>• Each instance on A is related to one and only one instance on B. |  <br>Many STUDENTs are enrolled in one COURSE, and each STUDENT is enrolled in one COURSE. | **Example:**<br>Many students take one course, but each student takes only one course.<br>  |
| **4. Many to many (N : N)** |  | • Many instances on A are related to many instances on B.<br>• Many instances on B are related to many instances on A. |  <br>Many STUDENTs are enrolled in many COURSES, and many COURSES have many STUDENTs. | **Example:**<br>Students can take many courses, and courses can have many students.<br>  |



## Week 5: Business Rules

### WHAT IS A BUSINESS RULE?

- A business rule is a guideline, policy, or requirement that tells an organization how something should be done.
- Simply: Business rules are the rules that a business follows when making decisions or performing activities.

### SIMPLE EXAMPLES

- **01:** A student must pay tuition before enrollment.
- **02:** An employee can only file leave if they have available leave credits.
- **03:** A customer must be at least 18 years old to apply for a loan.
- **04:** A product cannot be sold if its stock is 0.
- **05**: A loan cannot be approved if the applicant's income is below ₱30,000.

### WHY BUSINESS RULES ARE IMPORTANT

- Ensures consistency
- Improves decision making
- Supports smooth business operations
- Helps maintain compliance
- Increases efficiency

### HOW ARE BUSINESS RULES APPLIED IN DATABASES?

- Business rules are used to make sure that the data stored in a database follows the organization's rules.
- **Think of it this way:** *Business Rule* ➔ *Database Rule* ➔ *Valid / Invalid Data*

**EXAMPLE: STUDENT ENROLLMENT**

- **Business Rule:** A student cannot enroll if they have an unpaid balance.
- The database can enforce this rule by checking the student's balance before allowing enrollment.
- **1 IF**: `Balance = ₱0` ➔ `ENROLLMENT ALLOWED`
  - The system checks the balance. Since it is ₱0, enrollment is allowed.
- **2 IF:** `Balance = ₱5,000` ➔ `ENROLLMENT REJECTED`
  - The system checks the balance. Since it is greater than ₱0, enrollment is rejected.

### HOW BUSINESS RULES ARE APPLIED IN DATABASES

- **Primary Key:** Ensures each record is unique.
- **Not Null:** Ensures required data cannot be empty.
- **Check Constraint:** Ensures data follows specific conditions.
- **Foreign Key:** Ensures valid relationships between tables.
- **User Permissions:** Ensures only authorized users can access or change data.

### FORMULA-BASED BUSINESS RULES

**Definition**

- A formula-based business rule is a business rule that uses a mathematical formula or calculation to determine a value, result, or decision.
- Simple Definition: Formula-based rules use calculations to make business decisions or produce values automatically.

**EXAMPLE 1: STUDENT GRADE**

- **Business Rule:** Final Grade = 40% Midterm + 60% Final Exam
- **Calculation:** Final Grade = (85 × 40%) + (90 × 60%) = 34 + 54 = 88
- **Result:** Final Grade = 88

**EXAMPLE: SALES TABLE**

- **Business Rule:**
  - `Total Amount = Quantity × Unit Price`
  - `Discount Amount = Total Amount × Discount %`
  - `Net Amount = Total Amount – Discount Amount`
- **Key Formulas**
  - `Total Amount = Quantity × Unit Price`
  - `Discount Amount = Total Amount × Discount %`
  - `Net Amount = Total Amount – Discount Amount`
- **Key Takeaway:** Formula-based rules automatically calculate values based on data, ensuring accuracy and consistency.
- **In Summary:** Formula-based rules use calculations on data to generate values (like totals, discounts, balances). Databases can apply these rules using formulas, computed columns,

or queries to ensure accurate and consistent results.

- **Why is it important?:** It prevents invalid, inconsistent, or meaningless data from being stored in the database.

## HOW FORMULA-BASED BUSINESS RULES ARE APPLIED TO A DATABASE (SQL SERVER)

1. **Define The Business Rules (Formula-Based)**
   - `Total Amount = Quantity × Unit Price`
   - `Discount Amount = Total Amount × Discount %`
   - `Net Amount = Total Amount - Discount Amount`
2. **Create The Table With Formula-Based (Calculated) Columns**
   - `PERSISTED`**:** The calculated values are stored in the table and updated automatically whenever data changes.
3. **Insert Data (Only Enter The Base Values)**
   - You only insert the base data. The database automatically calculates the `TotalAmount`, `DiscountAmount`, and `NetAmount`.
4. **Result: Database Automatically Calculates The Values**
   - All calculated columns are updated automatically whenever data changes.
5. **How It Works**
   - Data is inserted (Quantity, UnitPrice, DiscountPercent) ➔ Database applies the formulas automatically ➔ Calculated columns generate the results (TotalAmount, DiscountAmount, NetAmount) ➔ Results are stored and always up-to-date whenever data changes

- **Key Benefits**
  - Ensures accuracy and consistency
  - Reduces manual computation
  - Automatically updates when data changes
  - Saves time and prevents errors
- **Note:** Use computed columns (AS ... PERSISTED) to apply formula-based rules in SQL Server.

## DATE/TIME-BASED BUSINESS RULE – SAMPLE APPLICATION IN DATABASE (SQL SERVER)

1. Business Rule (Date / Time Based)
   - If Payment Date > Due Date, charge a 5% late fee.
2. Database Table Design
   - **Table Name:** LOAN_PAYMENT
3. Create Table With Formula-Based Rule
4. Insert Data (Only Enter Base Values)
5. **Result:** Database Automatically Calculates The Values
   - Since PaymentDate > DueDate, the database automatically calculates the LateFee (5%) and TotalPayment.
6. How It Works
   - Base values are inserted ➔ Database checks dates and applies the business rule ➔ Calculated columns (LateFee, TotalPayment) are auto-computed ➔ Results are stored and always up-to-date when data changes

- Key Benefits
  - Ensures accuracy and consistency
  - Reduces manual computation
  - Automatically updates when dates change
  - Saves time and prevents errors
- **Note:** Use computed columns (AS ... PERSISTED) to apply date/time-based business rules in SQL Server.

## SCORING AND RATING – SAMPLE APPLICATION IN DATABASE (SQL SERVER)

1. **Business Rule (Formula-Based)**
   - **Final Score** = (Midterm × 40%) + (Final Exam × 60%)
   - Rating is based on the Final Score.
2. Database Table Design
   - **Table Name:** STUDENT_SCORE
3. Create Table With Formula-Based Rules (Sql Server)
4. Insert Data (Only Enter Base Values)
   - You only enter the Midterm and Final Exam scores. FinalScore and Rating are automatically computed by the database.
5. Result: Database Automatically Calculates The Values
   - FinalScore and Rating are computed automatically whenever scores are inserted or updated.

6. How It Works
    - Base scores (Midterm, Final Exam) are inserted ➔ Database applies the formula (0.40 Midterm + 0.60 Final Exam) ➔ Database applies the rating rules based on the Final Score ➔ Results are stored and always up-to-date

- **Key Benefits**
    - Ensures accuracy and consistency
    - Eliminates manual computation
    - Automatically updates when scores change
    - Saves time and prevents errors
- **Note:** Use computed columns (AS ... PERSISTED) to implement scoring and rating business rules in SQL Server.

## BUSINESS RULES - TYPES

**2 Type:** *Decision Table Based Rule*

- A decision table lists all possible combinations of conditions and the action (decision) that should be taken for each combination.
- **Example Scenario:** Determine the Final Grade Category based on: Midterm Score, Final Exam Score.
- **Note:** Decision table based rules help handle complex combinations of conditions in a clear and systematic way.

## DATABASE TABLE DESIGN

- **Table Name:** `STUDENT_GRADE`
- Create Table With Formula-Based Rules (Sql Server)
- Business Rules Applied
    - Scores must be between 0 and 100 (CHECK constraint).
    - FinalGrade is automatically computed based on the midterm and final exam scores (formula-based rule).
    - AS ... PERSISTED stores the computed value for performance and reporting.
- Note: The FinalGrade is automatically computed and cannot be updated manually.

## BUSINESS RULES - EXAMPLES

**In a corporate organization:**

- **Human Resources**: An employee must be at least 18 years old to be eligible for employment.
- **Example:** The system will not allow saving an employee record if the age is less than 18.
- **Finance:** All expenses must have an approved budget before payment.
    - **Example:** The system will not allow processing payment if the expense has no approved budget.
- **Procurement:** A purchase order must be approved by the department head if the amount is greater than ₱50,000.
    - **Example**: The system will require approval workflow for purchase orders above ₱50,000.
- **Inventory:** The stock quantity cannot be negative.
    - **Example:** The system will not allow issuing items if the stock on hand is 0 or less.
- **Sales:** A sales order cannot be created for a customer with an outstanding balance over ₱100,000.
    - **Example:** The system will block creating a new sales order if the customer balance exceeds ₱100,000.
- **Customer Service:** A complaint must be resolved within 3 working days.
    - **Example:** The system will escalate unresolved complaints after 3 working days.
- **IT / Security:** Users must change their password every 90 days.
    - **Example:** The system will prompt users to change their password after 90 days.

## BUSINESS RULES – HOW THEY ARE WRITTEN OR PRESENTED

*Noun - Verb - Noun | Term - Fact - Term*

- **Human Resources**
    - **Business Rule:** Employee - must be - at least 18 years old
    - **As a Term - Fact - Term:** Employment Eligibility - requires - age 18 or above
- **Finance**
    - **Business Rule:** Expense - must have - approved budget
    - **As a Term - Fact - Term:** Expense Payment - requires - approved budget
- **Procurement**
    - **Business Rule:** *Purchase Order - must be approved by - department head*

    - **As a Term - Fact - Term:** *High Value Purchase - requires approval by - department head*
- **Inventory**
    - **Business Rule:** *Stock Quantity - cannot be - negative*
    - **As a Term - Fact - Term:** *Inventory Balance - must not be - negative*
- **Sales**
    - **Business Rule:** *Sales Order - cannot be created for - customer with outstanding balance*
    - **As a Term - Fact - Term:** *Sales Order Creation - not allowed for - customer with outstanding balance*
- **Customer Service**
    - **Business Rule:** *Complaint - must be resolved within - 3 working days*
    - **As a Term - Fact - Term:** *Complaint Resolution - must be within - 3 working days*
- **IT / Security**
    - **Business Rule:** *User - must change - password every 90 days*
    - **As a Term - Fact - Term:** *Password Policy - requires - change every 90 days*

## IMPLEMENTATION OF BUSINESS RULES IN DBS

**Business rules will become constraints at the Database Level**

- An employee must be at least 18 years old to be eligible for employment. ➔ CHECK Constraint ➔ Ensures age is 18 or above. ➔ CHECK (Age >= 18)
- All expenses must have an approved budget before payment. ➔ CHECK Constraint ➔ Allows payment only if approved budget exists. ➔ CHECK (ApprovedBudget = 1)
- A purchase order must be approved by the department head if the amount is > ₱50,000. ➔ CHECK Constraint ➔ Requires approval if amount is greater than ₱50,000. ➔ CHECK (Amount <= 50000 OR ApprovedByDeptHead = 1)
- The stock quantity cannot be negative. ➔ CHECK Constraint ➔ Prevents saving negative stock. ➔ CHECK (StockQuantity >= 0)
- A sales order cannot be created for a customer with an outstanding balance over ₱100,000. ➔ CHECK Constraint ➔ Prevents sales order if balance is over ₱100,000. ➔ CHECK (OutstandingBalance <= 100000 OR IsSalesOrderAllowed = 0)
- A complaint must be resolved within 3 working days. ➔ CHECK Constraint ➔ Ensures resolution date is within 3 working days. ➔ CHECK (ResolutionDate <= DATEADD(DAY, 3, ComplaintDate))
- Users must change their password every 90 days. ➔ CHECK Constraint ➔ Ensures password age is not more than 90 days. ➔ CHECK (DATEDIFF(DAY, LastPasswordChange, GETDATE()) <= 90)
- Note: Business rules are enforced by constraints, triggers, stored procedures, and other database features to ensure data accuracy, consistency, and integrity at the database level.

## CONSTRAINTS

- Constraints are rules enforced by the database on the data in a table. They ensure accuracy, consistency, and reliability by restricting the type of data that can be stored.
- **Why are constraints important?**
    - **Maintain Data Accuracy**: Ensures only valid and correct data is stored.
    - **Ensure Data Consistency:** Prevents conflicting or duplicate data.
    - **Enforce Business Rules:** Implements company policies directly in the database.
    - **Improve Data Reliability:** Helps produce trustworthy reports and decisions.
- **Common Types of Constraints**
    - **Primary Key:** Uniquely identifies each record.
    - **Unique:** Ensures all values in a column are different.
    - **Not Null:** Ensures a column cannot have NULL values.
    - **Check:** Ensures values satisfy a specific condition.
    - **Foreign Key**: Ensures referential integrity between two tables.

## DATABASE INTEGRITY

- Database integrity ensures that data is accurate, consistent, and reliable throughout its lifecycle.
- **Accuracy:** Data is correct and reflects real-world values.
- **Consistency:** Data remains uniform and follows defined rules across the database.
- **Validity:** Data adheres to domain rules, constraints, and data types.
- **Constraints:** Rules like Primary Key, Foreign Key, Unique, Not Null, and Check enforce valid data.
- **Reliability:** Data is dependable and can be trusted for business operations and decision-making.
- **Audit Trail:** Changes are tracked and recorded to ensure accountability.
- **Benefits Of Database Integrity**
    - Improves data quality and trustworthiness
    - Supports accurate reporting and analysis
    - Enhances decision-making and operational efficiency
    - Reduces errors and data anomalies
    - Ensures compliance and regulatory standards
- Integrity is the foundation of trustworthy data.

## ENTITY INTEGRITY

- Entity Integrity states that every table must have a PRIMARY KEY, and each value in the primary key must be UNIQUE and NOT NULL.
- **Rules**
    1. **Primary Key** – every table must have its own primary key.
    2. **Unique** – no two rows can have the same primary key value.
    3. **Not Null** – primary key cannot contain NULL.
- **Invalid Examples**
    - **Duplicate Primary Key:** Not allowed: Duplicate value in primary key.
    - **Null Primary Key:** Not allowed: Primary key cannot be NULL.
    - **No Primary Key**: Not allowed: Table must have a primary key.
- **Key Takeaway:** Entity Integrity ensures that each record in a table is unique and identifiable using a primary key.

## ENTITY INTEGRITY (Application)

- Every table must have a PRIMARY KEY, and each value in the primary key must be UNIQUE and NOT NULL.
- **1. Create Table With Primary Key (Sql Server)**
    - **Explanation:** *StudentID is defined as PRIMARY KEY. SQL Server automatically enforces UNIQUE and NOT NULL for the primary key column.*
- **2. Insert Valid Data (Works)** – Using one query for multiple rows
- **3. Insert Invalid Data (Violates Entity Integrity)**
    - **A. Duplicate Primary Key (Not Unique):** Error: Violation of PRIMARY KEY constraint. Cannot insert duplicate key.
    - **B. NULL in Primary Key (Not Null):** Error: Cannot insert the value NULL into column. INSERT fails.
    - **C. Missing Primary Key (No Value):** Error: Column name or number of supplied values does not match table definition.
- **Key Takeaway:** Entity Integrity ensures that every record in a table is uniquely and identifiably represented using a PRIMARY KEY that is UNIQUE and NOT NULL.

## REFERENTIAL INTEGRITY

- A FOREIGN KEY in a child table must either match an existing PRIMARY KEY in the parent table, or be NULL (no relationship / unknown).
- **Valid References (Allowed):** Matches CustomerID in CUSTOMER table, or NULL is allowed (no relationship / unknown).
- **Invalid References (Not Allowed):** CustomerID does not exist in CUSTOMER table.
- **Why Orderid 104 Is Null?**
    - OrderID 104 has CustomerID = NULL because the customer is either:
        - Not yet known,
        - Does not exist in the CUSTOMER table,

        - Or the order is not linked to any customer (no relationship).
    - NULL is allowed by referential integrity because it represents "no relationship or unknown relationship".
- **Rule (Referential Integrity):**
    - **A Foreign Key value must either:**
        - Match an existing Primary Key in the referenced (parent) table, OR
        - Be NULL.
    - Any other value is not allowed.
- **Key Takeaway**: Referential Integrity keeps relationships between tables valid. It prevents "orphan" records that point to non-existent data.

## REFERENTIAL INTEGRITY (Application)

- A foreign key value must either match an existing primary key in the parent table, or be NULL.
- 1. Create Tables (SQL Server)
- 2. Insert valid data into Parent Table
- 3. Insert valid and NULL values into Child Table (Orders)
- 4 Try to insert an invalid foreign key
    - **Error:** The INSERT statement conflicted with the FOREIGN KEY constraint.
- 5 Other invalid examples
    - **Invalid:** duplicate or non-existing ID: Technically valid (since 2 exists), but if 106 already exists in Orders (and OrderID should be UNIQUE/PRIMARY KEY), it will fail due to primary key constraint, not foreign key.
    - **Invalid:** non-existing ID (99): Error: The INSERT statement conflicted with the FOREIGN KEY constraint.
    - **Valid:** NULL (no relationship): allowed because NULL means no relationship.
- **Key Takeaway:** Referential integrity keeps related tables consistent. A foreign key can refer to a valid primary key or be NULL. If it does not exist in the parent table (and is not NULL), the insert/update will be rejected.
    - 1 Valid references ➔ allowed
    - 2 NULL ➔ allowed (no relationship)
    - 3 Invalid references ➔ rejected
- ***Clean Data. Reliable Relationships. Stronger Databases.***

## DOMAIN INTEGRITY

- Domain Integrity means that all columns (attributes) in a relational database must have values from a DEFINED DOMAIN.
- **In short:** Each column has a set of allowed values (data type, range, format, or list).
- **Defined Domain (Rules For Each Column)**
    - `StudentID (INT):` Must be a whole number (integer).
    - `FullName (VARCHAR(50)):` Must be text, up to 50 characters.
    - `Age (INT):` Must be a whole number between 0 and 120.
    - `Gender (CHAR(1)):` Must be either 'M' or 'F'.
    - `Course (VARCHAR(20))`: Must be one of the allowed values: {'BSIT', 'BSCS', 'BA', 'BSA'}.
    - `Email (VARCHAR(100)):` Must be a valid email format.
- **Valid Data (Following The Domain Rules):** All values follow their defined domains.
- **Invalid Data (Violating The Domain Rules)**: e.g., Number not allowed, Age not in 0-120, Gender must be 'M' or 'F', Invalid email format, Exceeds 50 characters, Course not in allowed list, Not an integer.
- **Key Takeaway:** Domain Integrity ensures that every value in a column is VALID, CONSISTENT, and MEANINGFUL according to the rules (domain) defined for that column.
- **Why is it important?** It prevents invalid, inconsistent, or meaningless data from being stored in the database.

## DOMAIN INTEGRITY (Application)

- Domain Integrity means that all columns (attributes) in a relational database must have values from a DEFINED DOMAIN.
- **In Short:** Each column has a set of allowed values (data type, range, format, or list). Only values from that domain are allowed in the column.
- **1. Create Table (SQL Server):** Create table with defined domains for each column. CHECK constraints define the allowed domain for each column.
- **2. Insert Valid Data:** All values follow the domain rules.

- **3. Insert Invalid Data (Violates domain rules):**
    - **A. Age out of range (-5):**
        - **Error:** The INSERT statement conflicted with the CHECK constraint.
    - **B. Invalid Gender (X):**
        - **Error:** The INSERT statement conflicted with the CHECK constraint.
    - **C. Invalid Course (BSEE not in list):**
        - **Error:** The INSERT statement conflicted with the CHECK constraint.
    - **D. Invalid Email (wrong format):**
        - **Error:** The INSERT statement conflicted with the CHECK constraint.
- **4. Check Final Data:** Only valid rows are stored.
- **Key Takeaway:** Domain Integrity ensures every value in a column is VALID, CONSISTENT, and MEANINGFUL. Values that do not follow the defined domain rules will be rejected by the database.
- **Summary:**
    - Valid data ➔ allowed
    - Invalid data ➔ rejected
    - Consistency
    - Accuracy
    - Reliable data in your database

## WHAT IS A DOMAIN?

- A domain is the set of all allowed values for an attribute (column). It defines the data type, size, format, range, or list of values that can be stored in that column.
- **In simple words**: A domain is the "rule" that says what kind of data is allowed in a column.
- **Domains (Rules) For Each Column**
    - `StudentID (INT)`: Whole numbers only (0, 1, 2, 3, ...).
    - `FullName (VARCHAR(50))`: Text up to 50 characters.
    - `Age (INT)`: Whole numbers between 0 and 120.
    - `Gender (CHAR(1))`: Only 'M' or 'F'.
    - `Course (VARCHAR(20))`: Only one of these values: {'BSIT', 'BSCS', 'BA', 'BSA'}.
    - `Email (VARCHAR(100))`: Must be a valid email format.
- **Valid Data:** All values follow their defined domains.
- **Invalid Data:** Violates the domains (Numbers not allowed, Out of range, Must be 'M' or 'F', Invalid email format, Exceeds 50 characters, Not in allowed list, Not an integer).
- **Key Takeaway:** Domain Integrity ensures that every value in a column is VALID, CONSISTENT, and MEANINGFUL according to the domain (rules) defined for that column.
- **In Short:** A domain is the set of allowed values for a column — it controls the type, range, format, or list of data that can be stored.